

Deliivo Booking Request Expiry Feature

Developer Design Document - Rider-selected response deadline for Request-to-Book rides

Purpose

This document gives developers everything needed to implement the booking request expiry feature for riders and drivers: product behavior, business rules, UI wireframes, backend flow, APIs, database models, background jobs, payment handling, notifications, edge cases, and test scenarios.

Table of Contents

1. External reference and Poparide behavior
2. Product goal and v1 scope
3. Rider experience
4. Driver experience
5. Business rules
6. System architecture and services
7. Data model
8. API design
9. Background jobs and notifications
10. Payment behavior
11. Edge cases and concurrency
12. Test plan
13. Implementation checklist

1. External reference and Poparide behavior

Poparide supports a Request-to-Book model where a passenger sends a request, places a payment authorization, and the driver manually approves or declines. Passengers can set a booking request expiry time from 1 to 24 hours; if the driver does not respond in that timeframe, the request expires automatically. If the driver approves, the passenger payment is captured; if the driver declines, the passenger withdraws, or the request expires, the payment is not captured and the authorization is released.

Reference	Observed behavior to model
Poparide Help - Request to book	Passenger sets expiry time from 1 to 24 hours; driver must approve or decline in that timeframe.
Poparide Help - Respond to booking request	Driver navigates to the booking and chooses Approve or Decline; fast responses help passengers finalize travel plans.
Poparide Help - Expired request	Request expires if driver does not accept within selected time limit; card authorization is released automatically.
Poparide Terms	Request-to-book uses payment authorization first; capture happens only when driver approves; withdrawal before approval has no charge.

Deliivo v1 decision

Implement a similar request-to-book flow. The rider chooses a response deadline. The system pre-authorizes payment and holds the requested seats. Driver must approve or decline before expiry. If no response, an expiry worker closes the request and releases payment/seat hold.

2. Product goal and v1 scope

Feature goal: give riders control over how long they wait for driver approval, while giving drivers a clear response deadline and protecting seat/payment consistency.

In v1	Out of scope / v1.1
Rider selects response deadline: 1h, 3h, 6h, 12h, 24h, Before departure	Dynamic driver SLA pricing or penalties
Payment authorization at request time; capture on driver approval	Complex bidding or driver counter-offers
Seat hold while request is pending	Overbooking with manual driver choice between many riders
Driver approve/decline UI with countdown	Full driver reputation marketplace ranking
Auto-expiry worker + notifications	Advanced retry queue with dead-letter dashboards
Basic driver response metrics	Public quality-score display at launch

2.1 Recommended expiry options

User-facing option	Backend value	Validation
1 hour	expires_at = now + 1 hour	Allowed only if departure is more than now + 1 hour or cap to departure time
3 hours	expires_at = now + 3 hours	Must not exceed departure_time
6 hours	expires_at = now + 6 hours	Must not exceed departure_time
12 hours	expires_at = now + 12 hours	Must not exceed departure_time
24 hours	expires_at = now + 24 hours	Must not exceed departure_time
Before departure	expires_at = min(departure_time, now + 24 hours)	Never keep a request open more than 24h in v1

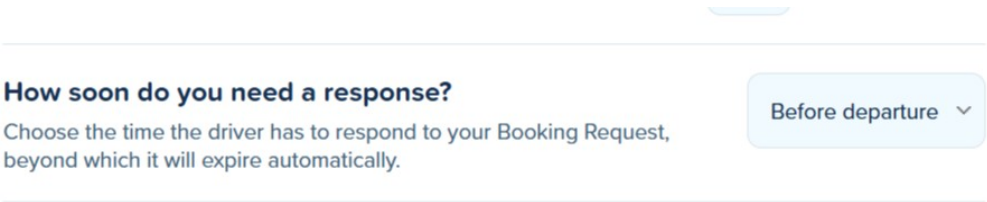
Default selection rule

Default expiry should depend on time to departure: <2h => 1h, 2-6h => 3h, 6-12h => 6h, 12-24h => 12h, >24h => 24h. This prevents unrealistic long pending requests for urgent rides.

3. Rider experience

3.1 Placement in booking flow

Show the response-deadline control on the final booking request confirmation screen, after seat count, pickup/drop-off, rider message, and payment method, before the rider sends the request.



Reference screenshot: rider chooses how soon they need a response.

3.2 Rider wireframe - Send booking request

```
+-----+
| Ride: Tallinn -> Tartu           1 seat selected |
+-----+
| Pickup      Tallinn Bus Station |
| Drop-off    Tartu Centre        |
| Message to driver [ Hi, I can meet 10 min early... ] |
| |
| How soon do you need a response? |
| Choose how long the driver has to respond to your |
| booking request. If they do not respond in time, |
| your request will expire automatically. |
| |
| [ 3 hours                               v ] |
| |
| Payment method |
| Visa ending 4242 |
| You will only be charged if the driver approves. |
| |
| [ Send booking request ] |
+-----+
```

3.3 Rider wireframe - Waiting for driver

```
+-----+
| Booking request sent |
+-----+
| Waiting for driver approval |
| Driver has until: Today 18:30 |
| Time left: 2h 45m |
| |
| Ride: Tallinn -> Tartu |
| Seats requested: 1 |
| Payment: Authorized, not charged yet |
| |
| [ Message driver ] [ Withdraw request ] |
| [ Find another ride ] |
+-----+
```

3.4 Rider states and labels

Backend status	Rider label	Allowed rider actions
----------------	-------------	-----------------------

PENDING_DRIVER_APPROVAL	Waiting for driver approval	Message driver, withdraw request, find another ride
APPROVED	Booking confirmed	View booking, message driver, cancellation rules apply
DECLINED	Driver declined	Find another ride
EXPIRED	Request expired	Find another ride, request again if trip still available
WITHDRAWN_BY_RIDER	You withdrew the request	Find another ride
PAYMENT_AUTH_FAILED	Payment authorization failed	Change payment method, retry
PAYMENT_CAPTURE_FAILED	Booking could not be confirmed	Retry with another payment method or send new request

4. Driver experience

4.1 Driver request detail wireframe

```

+-----+
| New booking request |
+-----+
| Rider: Ansul Sharma |
| Rating: 4.8   Phone verified   Payment authorized |
|
| Ride: Tallinn -> Tartu |
| Seats requested: 1 |
| Pickup: Tallinn Bus Station |
| Drop-off: Tartu Centre |
| Message: Hi, I can meet 10 minutes early. |
|
| Respond before: Today 18:30 |
| Time left: 2h 45m |
|
| [ Decline ] | [ Approve ] |
+-----+

```

4.2 Driver request list wireframe

```

+-----+
| Trip requests |
+-----+
| Tallinn -> Tartu, Today 20:00 |
|
| 1. Ansul - 1 seat - expires in 45m [Review] |
| 2. Riya - 2 seats - expires in 2h [Review] |
|
| Respond quickly to keep your response score high. |
+-----+

```

4.3 Driver actions

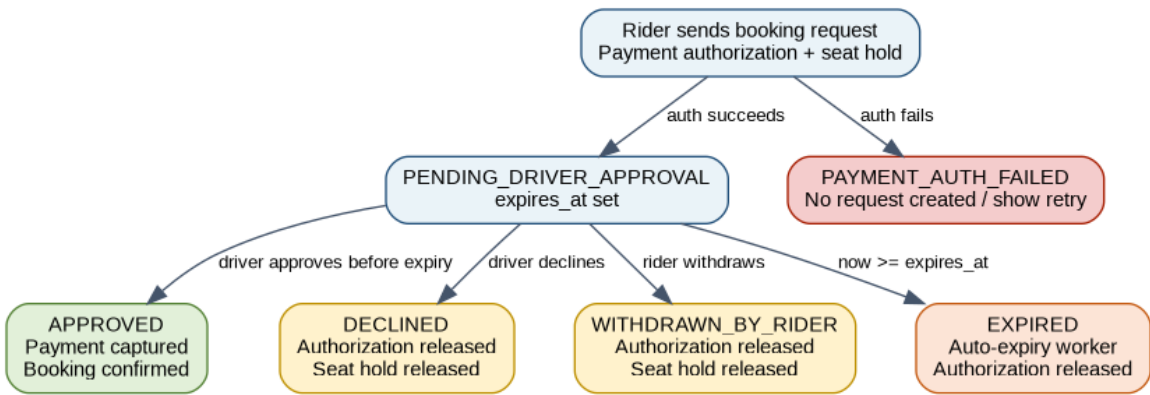
Action	System behavior	Driver result
Approve	Lock request row, validate not expired, validate seat hold, capture payment, create booking, convert hold, notify rider.	Booking confirmed; passenger appears in trip passenger list.
Decline	Lock request row, mark declined, release payment authorization, release seat hold, notify rider.	Request removed from pending list; rider sees declined.
No response	Expiry worker marks request expired, releases authorization and seat hold, notifies rider, updates response	Driver sees expired request in history.

	metrics.	
--	----------	--

5. Business rules

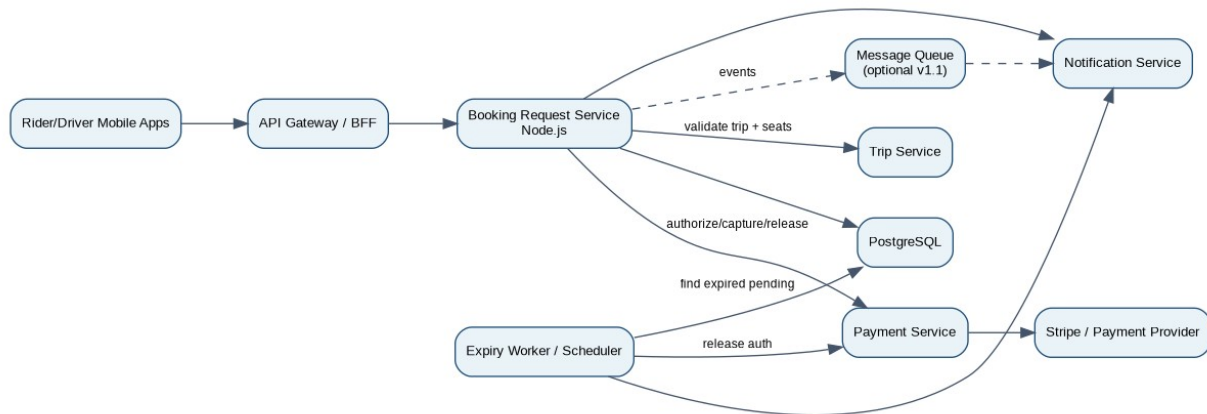
Rule area	Rule
Expiry min/max	Allowed request expiry is 1-24 hours. Before departure is internally capped to 24 hours or departure time, whichever is earlier.
Departure cap	expires_at must be <= trip.departure_time. A request cannot remain pending after trip departure.
Seat hold	Create an ACTIVE seat hold for seats_requested when request is created. Release it if declined, withdrawn, expired, or payment fails. Convert it if approved.
Pending capacity	In v1, do not allow pending requests to exceed available seats. This keeps driver approval simple and prevents overbooking.
Payment	Authorize at request time; capture only on approval; release on decline/withdraw/expiry.
Approval deadline	Driver cannot approve once now >= expires_at. Backend must reject even if app UI is stale.
Concurrency	Only one final action may win. Use database row locks and status checks.
Metrics	Expired pending requests count against driver response metrics; approvals and declines count as responses.

5.1 Booking request lifecycle



Lifecycle diagram: pending request can finish by driver approval, driver decline, rider withdrawal, payment failure, or automatic expiry.

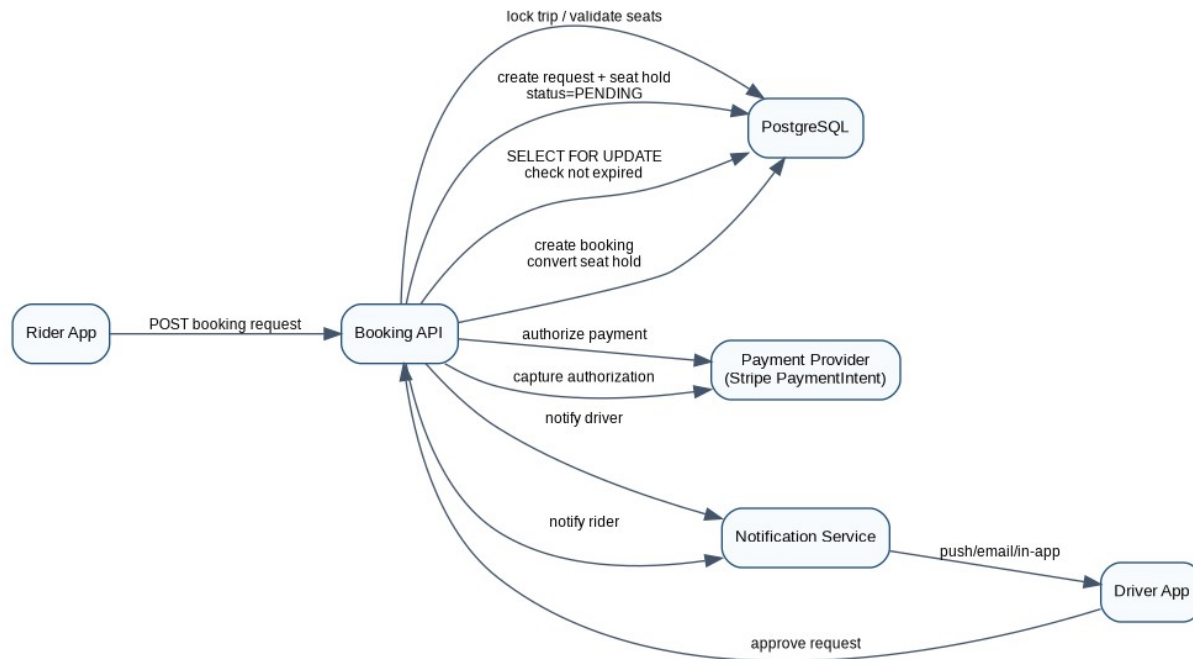
6. System architecture and services



Recommended service ownership for Node.js backend: Booking Request Service owns request lifecycle; Payment Service wraps payment provider; Trip Service owns trip and seat inventory; Notification Service sends push/email/in-app messages; Expiry Worker closes timed-out requests.

Component	Responsibilities
Mobile apps	Render expiry picker, countdowns, status screens, approve/decline actions, stale-state handling.
API Gateway / BFF	Auth, request routing, response shaping for rider/driver apps.
Booking Request Service	Create request, manage status transitions, approve/decline/withdraw, enforce rules.
Trip Service	Validate trip status, departure time, route stops, available seats.
Payment Service	Authorize, capture, release authorization, handle webhooks, idempotency.
Expiry Worker	Scheduled job that finds pending requests whose expires_at has passed and closes them.
Notification Service	Push/email/in-app notifications and retry of failed notifications.
PostgreSQL	Source of truth for booking requests, seat holds, bookings, audit logs, metrics.
Message Queue	Optional for v1; recommended for reliable async notifications, payment events, metrics updates.

6.1 Approval sequence



7. Data model

7.1 booking_requests

```
CREATE TABLE booking_requests (  
    id UUID PRIMARY KEY,  
    trip_id UUID NOT NULL,  
    rider_id UUID NOT NULL,  
    driver_id UUID NOT NULL,  
    seats_requested INT NOT NULL CHECK (seats_requested > 0),  
    pickup_stop_id UUID,  
    dropoff_stop_id UUID,  
    status VARCHAR(40) NOT NULL,  
    response_expiry_option VARCHAR(40) NOT NULL,  
    response_expiry_hours INT,  
    expires_at TIMESTAMPTZ NOT NULL,  
    rider_message TEXT,  
    decline_reason VARCHAR(100),  
    payment_authorization_id VARCHAR(255),  
    payment_status VARCHAR(40) NOT NULL,  
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
    approved_at TIMESTAMPTZ,  
    declined_at TIMESTAMPTZ,  
    expired_at TIMESTAMPTZ,  
    withdrawn_at TIMESTAMPTZ  
);
```

```
CREATE INDEX idx_booking_requests_pending_expiry  
ON booking_requests (expires_at)  
WHERE status = 'PENDING_DRIVER_APPROVAL';
```

```
CREATE INDEX idx_booking_requests_trip_status  
ON booking_requests (trip_id, status);
```

7.2 trip_seat_holds

```
CREATE TABLE trip_seat_holds (  
  id UUID PRIMARY KEY,  
  trip_id UUID NOT NULL,  
  booking_request_id UUID NOT NULL UNIQUE,  
  rider_id UUID NOT NULL,  
  seats_held INT NOT NULL CHECK (seats_held > 0),  
  status VARCHAR(30) NOT NULL,  
  expires_at TIMESTAMPTZ NOT NULL,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  released_at TIMESTAMPTZ  
);  
  
CREATE INDEX idx_trip_seat_holds_active  
ON trip_seat_holds (trip_id, status)  
WHERE status = 'ACTIVE';
```

7.3 Status enums

Enum	Values
booking_request.status	PENDING_DRIVER_APPROVAL, APPROVED, DECLINED, EXPIRED, WITHDRAWN_BY_RIDER, PAYMENT_AUTH_FAILED, PAYMENT_CAPTURE_FAILED, CANCELLED_DUE_TO_TRIP_CANCELLED
payment_status	AUTH_PENDING, AUTHORIZED, CAPTURED, AUTH_RELEASED, AUTH_FAILED, CAPTURE_FAILED
seat_hold.status	ACTIVE, RELEASED, CONVERTED_TO_BOOKING

8. API design

8.1 Create booking request - rider

POST /api/v1/trips/{tripId}/booking-requests
Authorization: Bearer <rider-token>

```
{  
  "seatsRequested": 1,  
  "pickupStopId": "uuid",  
  "dropoffStopId": "uuid",  
  "riderMessage": "Hi, I can meet 10 minutes early.",  
  "responseExpiryOption": "THREE_HOURS",  
  "paymentMethodId": "pm_123"  
}  
  
201 Created  
{  
  "bookingRequestId": "uuid",  
  "status": "PENDING_DRIVER_APPROVAL",  
  "expiresAt": "2026-06-05T18:30:00Z",  
  "paymentStatus": "AUTHORIZED",  
  "timeLeftSeconds": 9900  
}
```

8.2 Approve request - driver

POST /api/v1/booking-requests/{requestId}/approve
Authorization: Bearer <driver-token>
Idempotency-Key: <uuid>

200 OK

```
{
  "bookingRequestId": "uuid",
  "status": "APPROVED",
  "paymentStatus": "CAPTURED",
  "bookingId": "uuid"
}
```

Possible errors:

```
409 BOOKING_REQUEST_EXPIRED
409 BOOKING_REQUEST_NOT_PENDING
409 SEAT_HOLD_NOT_AVAILABLE
402 PAYMENT_CAPTURE_FAILED
403 DRIVER_NOT_OWNER_OF_TRIP
```

8.3 Decline request - driver

POST /api/v1/booking-requests/{requestId}/decline
Authorization: Bearer <driver-token>

```
{
  "reason": "PICKUP_NOT_SUITABLE"
}
```

200 OK

```
{
  "bookingRequestId": "uuid",
  "status": "DECLINED",
  "paymentStatus": "AUTH_RELEASED"
}
```

8.4 Withdraw request - rider

POST /api/v1/booking-requests/{requestId}/withdraw
Authorization: Bearer <rider-token>

200 OK

```
{
  "bookingRequestId": "uuid",
  "status": "WITHDRAWN_BY_RIDER",
  "paymentStatus": "AUTH_RELEASED"
}
```

8.5 Query endpoints

Endpoint	Used by	Purpose
GET /api/v1/riders/me/booking-requests? status=PENDING_DRIVER_APPROVAL	Rider app	Show active waiting requests and countdown.
GET /api/v1/drivers/me/booking-requests? status=PENDING_DRIVER_APPROVAL	Driver app	Show pending requests sorted by expires_at ASC.
GET /api/v1/booking-requests/{requestId}	Both	Show full request detail with allowed actions.

9. Background jobs and notifications

9.1 Expiry worker

```
// Run every 1 minute in v1. Use distributed lock if multiple worker instances exist.
async function expirePendingBookingRequests() {
  const requests = await db.query(`
    SELECT * FROM booking_requests
    WHERE status = 'PENDING_DRIVER_APPROVAL'
    AND expires_at <= now()
    ORDER BY expires_at ASC
    LIMIT 500
    FOR UPDATE SKIP LOCKED
  `);

  for (const request of requests) {
    await db.transaction(async (tx) => {
      const locked = await tx.bookingRequests.lockById(request.id);
      if (locked.status !== 'PENDING_DRIVER_APPROVAL') return;
      await paymentService.releaseAuthorization(locked.payment_authorization_id);
      await tx.seatHolds.releaseByBookingRequestId(locked.id);
      await tx.bookingRequests.markExpired(locked.id);
      await tx.driverMetrics.recordExpired(locked.driver_id);
      await notificationService.notifyRiderRequestExpired(locked.id);
      await notificationService.notifyDriverRequestExpired(locked.id);
    });
  }
}
```

Notification trigger	Recipient	Message example
Request created	Driver	New booking request. Please respond before 18:30 today.
Reminder before expiry	Driver	Booking request expires in 1 hour.
Driver approved	Rider	Your booking is confirmed.
Driver declined	Rider	The driver declined your booking request. You were not charged.
Request expired	Rider	Your request expired because the driver did not respond in time.
Rider withdrew	Driver	The rider withdrew the booking request.
Payment capture failed	Rider and driver	Booking could not be confirmed due to payment issue.

10. Payment behavior

Lifecycle event	Payment action	Seat action
Rider sends request	Create authorization / PaymentIntent with manual capture	Create ACTIVE seat hold
Driver approves	Capture authorized amount	Convert hold to confirmed booking
Driver declines	Cancel/release authorization	Release hold
Rider withdraws	Cancel/release authorization	Release hold
Request expires	Cancel/release authorization	Release hold
Capture fails	Mark PAYMENT_CAPTURE_FAILED; release/cancel authorization if possible	Release hold

Stripe implementation hint

Use PaymentIntent with capture_method=manual for authorization. Store payment_intent_id as payment_authorization_id. Capture on approval. Cancel the PaymentIntent on decline, withdrawal, or expiry. Add idempotency keys to create, capture, and cancel calls.

11. Edge cases and concurrency

Edge case	Required behavior
Driver approves after expiry	Reject with BOOKING_REQUEST_EXPIRED. Release stale hold if needed.
Rider withdraws while driver approves	Use SELECT FOR UPDATE; whichever transaction commits first wins.
Payment auth succeeds but notification fails	Do not rollback request. Retry notification async.
Driver changes departure time	If new departure time is earlier than expires_at, update expires_at and notify rider/driver.
Driver cancels trip while request pending	Mark request CANCELLED_DUE_TO_TRIP_CANCELLED; release authorization and seat hold.
Trip becomes full after stale UI	Create request endpoint must validate capacity using latest database state.
Payment capture fails after driver approves	Do not create confirmed booking. Mark CAPTURE_FAILED and notify both sides.
Server down during expiry time	Expiry worker should process overdue requests when service recovers. Do not rely only on in-memory timers.
Duplicate approve tap	Use idempotency key and status check. Return existing approved booking if already approved.
Timezone mismatch	Store timestamps in UTC/TIMESTAMPZ. UI formats in local trip timezone.

```
// Critical transaction pattern for final-state actions
await db.transaction(async (tx) => {
  const request = await tx.queryOne(`
    SELECT * FROM booking_requests WHERE id = $1 FOR UPDATE
  `, [requestId]);

  if (request.status !== 'PENDING_DRIVER_APPROVAL') throw new ConflictError();
  if (request.expires_at <= now()) throw new ExpiredError();

  // Continue with capture/decline/withdraw logic...
});
```

12. Test plan

Test category	Scenarios
Unit tests	expiry calculation; default option selection; status transition validation; capacity calculation including active holds.
API tests	create request success/failure; approve; decline; withdraw; stale approve; unauthorized driver/rider.
Payment tests	auth success; auth failure; capture success; capture failure; cancel/release called on every non-approved final state.
Worker tests	expires due requests; ignores non-pending; handles payment release error; idempotent re-run.
Concurrency tests	approve vs withdraw race; approve vs expiry race; multiple riders request last seat.
UI tests	expiry dropdown; countdown; expired state; disable approve after expiry; refresh stale screen.
Integration tests	full rider request -> driver approve; request -> expire; request -> decline; request -> withdraw.

13. Implementation checklist

Backend

- Add booking_requests and trip_seat_holds tables
- Add status enums and repository methods
- Implement create/approve/decline/withdraw APIs
- Add transaction locks for final-state actions
- Implement expiry worker with FOR UPDATE SKIP LOCKED
- Add idempotency keys for payment operations
- Add audit logs for every status transition

Payments

- Authorize at request creation
- Capture only on approval
- Release authorization on decline/withdraw/expiry
- Handle capture failure safely
- Add webhook handler to reconcile provider state

Rider app

- Add response deadline dropdown
- Show payment authorization note
- Show pending request countdown
- Implement withdraw request flow
- Show expired/declined/approved states

Driver app

- Show pending request list sorted by expiry
- Show request detail with countdown
- Implement approve and decline actions
- Disable actions after expiry
- Show expired request history

Notifications

- New request to driver
- Driver reminder before expiry
- Approved/declined/expired to rider
- Withdrawn to driver
- Payment failure to both sides

Observability

- Metrics: pending count, expired count, approval latency, payment capture failures
- Logs: request id, trip id, actor id, old/new status
- Alerts: expiry worker failures and payment release failures

Sources used for Poparide reference

- Poparide Help Centre, "What does Request to book mean?", <https://support.poparide.com/en/articles/9702658-what-does-request-to-book-mean>
- Poparide Help Centre, "How to respond to a booking request", <https://support.poparide.com/en/articles/9702665-how-to-respond-to-a-booking-request>
- Poparide Help Centre, "Why did my booking request expire?", <https://support.poparide.com/en/articles/13641664-why-did-my-booking-request-expire>

- Poparide Terms of Service, Request-to-book payment authorization and expiry behavior, <https://www.poparide.com/en-ca/terms>